ACADEMIC PROJECT REPORT

IMovies

Movie Ticket Booking Web Application

Full Stack Development (FSD) Course Documentation

Project Identification & Metadata

Project Title:	IMovies – Online Movie Ticket Reservation System
Project Type:	Full Stack Development College Project
Frontend Stack:	React.js, Vite Build Tool, JavaScript, CSS
Backend Stack:	Node.js, Express.js REST Framework
Database:	MongoDB (Local instance: mongodb://127.0.0.1:27017/imovies)
Security:	JWT (JSON Web Tokens) with localStorage Session Handling
Source Control:	https://github.com/nandanig18/IMovies

Academic Submission Details

Student Name:	`[Enter Student Full Name]`
Roll / Enrollment No:	`[Enter University Enrollment Number]`
Program & Branch:	`[e.g., B.Tech / BCA / MCA - Computer Science]`
Semester / Year:	`[e.g., 6th Semester / 3rd Year]`
Institution:	`[Enter College / University Name]`
Faculty Mentor / Guide:	`[Enter Guide Name & Designation]`
Date of Submission:	`[Enter Submission Date: DD/MM/YYYY]`

Submitted in partial fulfillment of the academic requirements for the Full Stack Development Laboratory coursework.

Table of Contents

1. Introduction	Page 2	8. Authentication	Page 5
2. Project Overview	Page 2	9. User Interface	Page 5
3. Architecture	Page 3	10. Testing	Page 5
4. Setup Instructions	Page 3	11. Screenshots & Interface Wireframes	Page 6
5. Folder Structure	Page 4	12. Known Issues & Operational Limits	Page 6
6. Running the Application	Page 4	13. Future Enhancements & Conclusion	Page 6
7. API Documentation	Page 4		

1. Introduction

The **IMovies** project is an interactive movie ticket reservation web platform developed as an academic showcase for modern Full Stack Development (FSD). The primary objective of the application is to demonstrate end-to-end client-server interaction, dynamic UI state management, relational-like transactional persistence in a NoSQL environment, stateless user session verification via JSON Web Tokens (JWT), and the execution of fundamental Create, Read, and Delete (CRUD) operations.

By decoupling the presentation layer from backend business logic through standardized RESTful API endpoints, IMovies implements a modern architectural pattern. The system addresses common functional workflows in e-ticketing systems, including catalogue exploration, real-time interactive seat selection, transactional booking registration, and booking lifecycle management.

2. Project Overview

IMovies enables authenticated users to browse movie schedules, review seat availability, compute ticket fares based on dynamic seat counts, complete simulated ticket checkouts, and maintain records of their personal booking history with self-service cancellation capability.

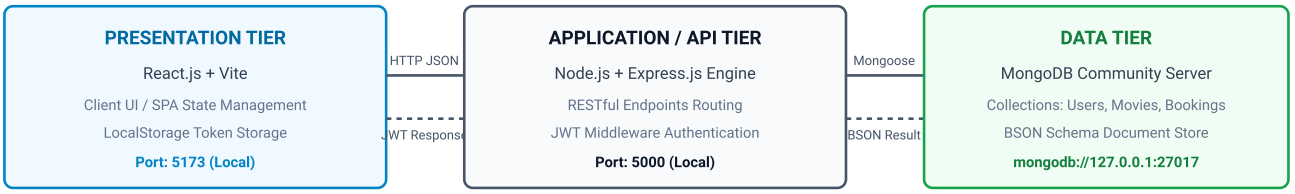
> **Academic Scope & Demonstration Goals:**
> - **Client-Side Engineering:** Building dynamic, single-page application (SPA) views using React and Vite.
> - **Server-Side Logic:** Developing modular Express routes to handle data serialization and JSON payloads.
> - **Database Integration:** Implementing asynchronous document storage with local MongoDB instances.
> - **Security Implementation:** Demonstrating credential hashing and token-based route authorization.

Summary of Core Features

Feature Module	Functional Description	Implementation Technique
User Registration & Login	Onboards new accounts and validates existing credentials to issue cryptographic tokens.	Password submission, JWT generation, local client persistence.
Catalogue Retrieval	Displays active cinematic titles including *Inception*, *3 Idiots*, and *Dangal*.	Asynchronous fetch through backend REST endpoints.
Interactive Seat Grid	Allows users to toggle seats and calculate aggregate price at ₹150 per seat.	React functional state hooks (useState) tracking seat indices.
Simulated Checkout	Conducts simulated transaction processing without external monetary exchange.	Demo payment screen emitting confirmation payloads.
My Bookings & Cancellation	Shows user-specific booking history with immediate record deletion.	HTTP GET and DELETE queries synchronizing database records.

3. Architecture

The IMovies web application adheres to a decoupled, 3-tier client-server architectural model. This ensures a clean separation of concerns among presentation, business routing, and document persistence:



- **Frontend Layer:** Employs React components with functional hooks. Client-side HTTP requests transmit JSON payloads and attach authorization headers.
- **Backend Layer:** Node/Express coordinates CORS, payload body parsing, route distribution, and JWT token encryption/validation.
- **Data Layer:** MongoDB persists application documents. Local access is configured via the default connection uri: `mongodb://127.0.0.1:27017/imovies`.

4. Setup Instructions

To recreate the development runtime locally, follow this standardized step-by-step setup procedure. Ensure you have installed **Node.js (v16+ recommended)**, **npm**, and the **MongoDB Community Server**.

Step	Action & Console Command	Operational Purpose
Step 1	`git clone https://github.com/nandanig18/IMovies.git` `cd IMovies`	Clones the project code from GitHub into your local working directory.
Step 2	`cd backend` `npm install`	Installs backend dependencies (Express, Mongoose, JsonWebToken, Dotenv, Cors, etc.).
Step 3	`cd ../frontend` `npm install`	Installs frontend dependencies (React runtime, Vite bundling tools, router utilities).

Step 4	Create `backend/.env` file:
	`PORT=5000`
	`MONGO_URI=mongodb://127.0.0.1:27017/imovies`
	`JWT_SECRET=[developer_local_secret]`
		Sets local environment variables. Note: Never commit private production keys or sensitive secrets to repository tracking.

Step 5	Verify local MongoDB Daemon: `mongod --dbpath /data/db` (or Windows Service)	Ensures the database daemon is listening on local port 27017 prior to backend initialization.

5. Folder Structure

The IMovies codebase is structured as a decoupled monorepo containing distinct directories for client and server logic:

```
      IMovies/ ├── backend/ │ ├── config/ # Database connection setup (MongoDB Mongoose
driver) │ ├── controllers/ # Controller handlers for business logic (Auth, Movies, Bookings) │
├── models/ # Mongoose database schemas (User, Movie, Booking) │ ├── routes/ # Express route
definitions mapping HTTP verbs to controllers │ ├── middleware/ # JWT verification and route
protection middlewares │ ├── .env # Local environment configurations (private; ignored in Git)
│ ├── server.js # Server entry point initializing Express, CORS, and port listening │ └──
package.json # Backend dependencies and scripts └── frontend/ ├── public/ # Static assets and
movie placeholder graphic resources ├── src/ │ ├── components/ # Reusable UI components
(Navbar, SeatGrid, MovieCard) │ ├── pages/ # View components (Home, Movies, Bookings, Login,
Register, Payment) │ ├── context/ # State management / Auth state provider (optional/hook
based) │ ├── App.jsx # Top-level React router composition │ ├── main.jsx # Vite client
mounting entry point │ └── index.css # Global stylesheet definitions ├── index.html # Vite
HTML shell ├── vite.config.js # Vite server settings and build optimizations └── package.json
# Frontend dependencies and dev scripts
```

6. Running the Application

Execution requires opening two separate terminal instances in Visual Studio Code or your local shell:

- **Terminal 1 (Backend API Service):** Execute `cd backend && npm start` (or `node server.js`). The service starts at `http://localhost:5000/`.
- **Terminal 2 (Frontend Client Service):** Execute `cd frontend && npm run dev`. The Vite local development server mounts at `http://localhost:5173/`.

> **Runtime Boundary Notice:** The local URLs `http://localhost:5173/` and `http://localhost:5000/` are hosted exclusively within the local host loopback network. They are accessible only while both background node instances remain active.

7. API Documentation

The backend serves RESTful JSON endpoints. Below are the verified API routes implemented in the current build:

Method	Endpoint URL	Payload / Params	Description
GET	`/api/movies`	None	Fetches available movies (Inception, 3 Idiots, Dangal).
GET	`/api/bookings`	Headers: `Authorization`	Retrieves past bookings for the authenticated user session.
DELETE	`/api/bookings/:id` *	URL param: booking record ID	Deletes an existing booking record from the database.
POST	`/api/auth/register` *	`{ name, email, password }`	Registers a new user record.
POST	`/api/auth/login` *	`{ email, password }`	Authenticates credentials and returns a signed JWT.

** Note: Exact auth route paths and parameter formats are based on standard Express naming patterns and are subject to verification against specific controller files in the repository.*

8. Authentication

User authentication is handled via JSON Web Tokens (JWT) using a stateless token-based approach:

- **Credential Validation:** During login, user credentials are confirmed against database records.
- **Token Issuance:** Upon successful verification, the Express server signs a token using the server's private `JWT_SECRET`, packaging non-sensitive payload claims (such as user ID).
- **Client Persistence:** The React application saves the returned JWT in `localStorage`, allowing persistent logins across page reloads.
- **Protected Requests:** Authenticated calls to protected routes (e.g., retrieving or canceling bookings) attach the token in the HTTP request headers (`Authorization: Bearer <token>`).

9. User Interface

The user interface is designed as a responsive Single-Page Application (SPA) with straightforward, accessible navigation:

- **Authentication Views:** Registration and Login screens with structured input fields and client-side validation.
- **Movies Showcase:** A catalog grid displaying movie cards with titles, poster placeholders, and details for *Inception*, *3 Idiots*, and *Dangal*.
- **Interactive Seat Layout:** A theater-style seat grid where users select individual seats. The calculated total updates in real time at ₹150 per seat.
- **Demo Payment View:** A mock checkout screen featuring a "Pay Now" trigger. Selecting it produces a successful confirmation state without transferring real currency.
- **My Bookings Dashboard:** A personal history log that allows users to review confirmed tickets or cancel them directly.

10. Testing

All completed features were manually tested in a local development environment. Below are the verified test results:

Test Case	Expected Result	Actual Result (Local Test)	Status
TC-01: Movie Catalogue	Movies page displays available movie cards via backend API.	API returned Inception, 3 Idiots, and Dangal correctly; cards rendered in grid.	**PASS (Local)**
TC-02: Seat Selection	Seats toggle state on click and update total fare at ₹150 per seat.	Selected seats highlighted correctly; aggregate price calculated as expected.	**PASS (Local)**
TC-03: Checkout Navigation	Clicking "Confirm Booking" routes user to the payment screen.	Application successfully navigated to the mock payment interface.	**PASS (Local)**
TC-04: Demo Payment	Clicking "Pay Now" shows a payment success notification.	Mock confirmation message appeared without processing any real payments.	**PASS (Local)**
TC-05: Booking History	"My Bookings" page lists current and past user reservations.	Booking details retrieved from MongoDB and displayed in history view.	**PASS (Local)**
TC-06: Booking Cancellation	Canceling a reservation removes it from the user's booking list.	Record was deleted from database; UI list refreshed immediately.	**PASS (Local)**

> **Verification Scope Note:** These results reflect manual verification performed on local development servers (Node.js and MongoDB on localhost). They do not represent automated testing or production deployment results.

11. Screenshots or Demo

The following wireframe diagrams illustrate the primary user workflows in the local application. Replace these placeholders with your actual screen captures prior to final submission:

[Figure 11.1: Movies Catalogue & Selection]

Shows movie titles (Inception, 3 Idiots, Dangal) alongside poster cards and selection buttons.

[Figure 11.2: Interactive Seat Layout]

Interactive cinema seat grid displaying selected seats and running total (Seats × ₹150).

[Figure 11.3: Demo Payment Modal]

Mock payment checkout window featuring the "Pay Now" button and success notification.

[Figure 11.4: My Bookings & Cancellation]

Personal booking history dashboard displaying confirmed tickets and the "Cancel Booking" action.

12. Known Issues & Operational Limits

The current project build has the following known operational constraints and technical limitations:

- **Deployment Incomplete:** The application is not yet deployed online. An earlier deployment attempt on Render was postponed due to pending email verification. A public deployment for the frontend, backend, and cloud database remains pending.
- **Mock Payment Gateway:** The checkout flow is entirely simulated for academic demonstration. No actual payment provider (e.g., Stripe, Razorpay) is integrated, and no real-money transactions are processed.
- **Local Host Dependency:** Because the application is not publicly hosted, the system functions only when local frontend (port 5173), backend (port 5000), and MongoDB (port 27017) processes are running concurrently.
- **Seat Concurrency:** Seat availability is managed locally without distributed locking, meaning concurrent bookings by multiple users for the same seat are not synchronized in real time.

13. Future Enhancements & Conclusion

The IMovies project successfully demonstrates the fundamental principles of full-stack web development using the MERN stack (MongoDB, Express, React, Node.js). To evolve this academic prototype into an enterprise-ready platform, several key enhancements are planned:

- **Cloud Deployment:** Complete platform deployment by provisioning an online database cluster (MongoDB Atlas) and deploying services to cloud hosts like Render or Vercel.
- **Production Payment Gateway:** Integrate a certified payment sandbox (such as Razorpay or Stripe) using webhook validation for real-time transactions.
- **Real-Time Seat Synchronization:** Implement WebSockets (Socket.io) to support live seat locking and prevent duplicate bookings across concurrent user sessions.
- **Ticketing Automation:** Add automated booking confirmations with dynamic QR code generation and email notifications via Nodemailer.